快捷选项

后端文件夹:

```
cd /d C:\Users\Gao\Desktop\毕业设计论文和方案\IoT_System
```

启动:

```
python manage.py runserver
```

前端文件夹:

```
cd /d C:\Users\Gao\Desktop\毕业设计论文和方案\IoT_System\frontend
```

```
npm run dev
```

模拟数据:

🟢 环境配置

验证 Node.js (检查你之前装的前端环境)：node -v

验证 Python (这是 Anaconda 自带的，肯定没问题)：python --version

第一步：进入你的目标文件夹

```
cd /d C:\Users\Gao\Desktop\毕业设计论文和方案\IoT_System
`mkdir` (Make Directory) = 创建文件夹；`cd` = 进入文件夹。
```

第二步：创建代码专用的子文件夹

```
mkdir IoT_System
cd IoT_System
```

第三步：安装环境 & 创建后端

安装 Django 并创建项目

```
pip install django django-cors-headers requests

Django: 一个超强大的 Web 框架。它帮我们处理了 80% 的重复工作（如怎么连数据库、怎么处理用户请求）。
```

```
django-cors-headers: "CORS" 是跨域资源共享。简单说，为了安全，浏览器默认不允许网页 A 访问服务器 B。这个插件就是给安保人员的“通行证”，允许你的前端网页访问你的后端数据。

requests: 一个让 Python 能上网发请求的工具。比如你的物联网设备要发数据，或者你要调用别人的天气 API，就靠它。它是餐厅的电话机。
```

创建后端骨架

```
django-admin startproject core .
召唤 Django 管理员，开始一个叫 core 的新项目。注意最后的 . (点) 非常重要!
这是生成项目的总控室。core 文件夹里会包含 settings.py（配置文件）和 urls.py（路由表）。
如果不加点：Django 会创建一个文件夹叫 core，里面再套一个 core，层级太深，很难看。
```

创建功能模块:

```
python manage.py startapp device_api
让manage.py 创建一个名为 device_api 的应用 (App)
```

后端启动

```
python manage.py runserver
```

停止服务器

```
Ctrl + C
```

迁移数据库

```
python manage.py migrate

Django 自带了一个强大的数据库管理系统。

虽然你还没写任何代码，但 Django 自带了“用户管理”、“权限管理”等功能。

这些功能需要数据库里有对应的表 (Table)。
```

🟢 第一阶段：注册应用与设计数据库

第一步：修改配置文件 (settings.py)

打开文件夹 `core/` 下的 `settings.py` 文件。

1. 找到 `INSTALLED_APPS` (大约在第 33 行)

这里列出了所有启用的功能。我们需要把刚安装的 `corsheaders` (跨域插件) 和我们创建的 `device_api` 加进去。

请修改成下面这样（注意最后的逗号）：

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    # --- 新增下面这两行 ---
    'corsheaders',      # 允许跨域请求的插件
    'device_api',       # 我们自己创建的 APP
]
```

2. 找到 `MIDDLEWARE` (大约在第 44 行)

确保它能第一时间拦截并处理请求。

请修改成下面这样（在列表最顶端添加一行）：

```
MIDDLEWARE = [
    # --- 新增这一行，必须放在最上面 ---
    'corsheaders.middleware.CorsMiddleware',

    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]
```

3. 在文件最底部添加 CORS 配置

为了方便开发，我们允许任何来源（比如你电脑上的 React 网页）访问这个后端。

在 `settings.py` 文件的最后一行，按几下回车，粘贴下面这行代码：

```
# 允许所有来源访问（开发模式专用）
CORS_ALLOW_ALL_ORIGINS = True
```

第二步：设计数据库 (models.py)

我们要设计一张表来存传感器数据。 打开 `device_api/` 文件夹下的 `models.py` 文件。 把它原本的内容清空，复制粘贴下面的代码：

```
from django.db import models+


# Create your models here.
```

```
# 想象我们在设计一张 Excel 表格，表名叫 SensorData
class SensorData(models.Model):
    # 第一列：温度 (浮点数，比如 25.5)
    temperature = models.FloatField()

    # 第二列：湿度 (浮点数，比如 60.2)
    humidity = models.FloatField()

    # 第三列：时间 (自动记录当前时间，不需要手动填)
    timestamp = models.DateTimeField(auto_now_add=True)

    # 这是一个方便给人类看的功能
    # 当你在后台查看数据时，它会显示 "时间 - 温度" 这样的格式，而不是 "Object(1)"
    def __str__(self):
        return f"{self.timestamp} - Temp: {self.temperature}"
```

第三步：生成并应用数据库 (终端操作)

到你的黑色命令行窗口

1. 生成图纸 (Make Migrations)

告诉 Django："我修改了 models.py，请帮我生成一份数据库施工图纸。"

```
python manage.py makemigrations
```

如果成功，你会看到提示：`Create model SensorData`

第二阶段：编写接口 (Views) 与 路由 (URLs)

这一步我们要打通"数据进"和"数据出"的通道。

第一步：编写业务逻辑 (views.py)

`views.py` 负责接收，处理后再返回结果。

打开 `device_api/views.py`，**清空**里面的内容，粘贴以下代码：

```
from django.http import JsonResponse
from django.views.decorators.csrf import csrf_exempt
import json
from .models import SensorData

# ------------------------------------------------------------
# 接口 1: 接收数据 (专门给 ESP32 用)
# ------------------------------------------------------------
# @csrf_exempt 的意思是：允许"陌生人"发请求。
# 因为 ESP32 只是个芯片，没有浏览器的安全证书，不加这个它发不进来。
@csrf_exempt
def receive_data(request):
    if request.method == 'POST':
        try:
            # 1. 拆开 ESP32 发来的包裹 (JSON数据)
            data = json.loads(request.body)
```

```
            # 2. 把数据写入刚才设计的数据库表格中
            SensorData.objects.create(
                temperature=data['temperature'],
                humidity=data['humidity']
            )

            # 3. 回复 ESP32: "收到了！"
            return JsonResponse({"status": "success", "message": "Data saved!"})

        except Exception as e:
            # 如果出错 (比如发来的数据格式不对)，报错给它看
            return JsonResponse({"status": "error", "message": str(e)})

    return JsonResponse({"status": "error", "message": "Only POST allowed"})

# ---------------------------------------------------------
# 接口 2: 获取历史数据 (给 React 网页用)
# ---------------------------------------------------------
def get_history(request):
    # 1. 去数据库捞数据：按时间倒序(最新的在前面)，只取最近10条
    raw_data = SensorData.objects.all().order_by('-timestamp')[:10]

    # 2. 整理数据：把数据库对象转换成简单的字典列表
    result_list = []
    for item in raw_data:
        result_list.append({
            "temp": item.temperature,
            "humi": item.humidity,
            "time": item.timestamp.strftime("%H:%M:%S") # 把时间转成 "12:30:05" 这
种格式
        })

    # 3. 把整理好的列表发回给网页
    return JsonResponse({"data": result_list})
```

第二步：配置子路由 (创建 device_api/urls.py)

Django 默认没有给 APP 创建路由文件，我们需要自己新建一个。 **请在 `device_api/` 文件夹下，右键 -> 新建文件 -> 命名为 。**

在里面粘贴以下代码：

```
from django.urls import path
from . import views

# 这里定义具体的门牌号
urlpatterns = [
    # 如果访问 /api/upload/ -> 就找 views.receive_data 干活
    path('upload/', views.receive_data),

    # 如果访问 /api/history/ -> 就找 views.get_history 干活
    path('history/', views.get_history),
]
```

第三步：配置总路由 (修改 core/urls.py)

现在子路由建好了，要把它挂载到主路由上。 打开 `core/urls.py`，**修改成下面这样**：

```
from django.contrib import admin
from django.urls import path, include  # <--- 注意：这里要多导入一个 include

urlpatterns = [
    path('admin/', admin.site.urls),

    # 意思是：凡是以 'api/' 开头的网址，都转交给 device_api 去处理
    path('api/', include('device_api.urls')),
]
```

第四步：最终测试

现在代码都写好了，我们来验证一下服务器能不能跑通。

1. **启动服务器** (在终端输入)：

   ```
   python manage.py runserver
   ```

2. **验证**： 打开你的浏览器 (Chrome/Edge)，在地址栏输入：`http://127.0.0.1:8000/api/history/`

3. **预期结果**： 你应该会看到类似这样的文字（虽然现在是空的，因为还没存数据）：

   ```
   {"data": []}
   ```

🟢 第三阶段：模拟设备发送数据 (验证 POST 接口)

第一步：创建模拟脚本

请在 `IoT_System` 文件夹下（也就是能看到 `manage.py` 的那个地方），新建一个文件，命名为 `simulate_device.py`。

注意：这个文件是独立的，它不属于 Django，它只是一个测试工具。

复制以下代码进去（已修复 URL 格式问题）：

```
import requests
import random
import time

# 修正：去掉了多余的 markdown 符号，只保留纯净的网址
url = "http://127.0.0.1:8000/api/upload/"
print("开始模拟设备发送数据... (按 Ctrl+C 停止)")

while True:
    # 1. 生成随机的温度和湿度
    # random.uniform(20, 30) 意思是产生一个 20 到 30 之间的随机小数
```

```
    temp = round(random.uniform(20.0, 35.0), 1)
    humi = round(random.uniform(40.0, 60.0), 1)

    # 2. 打包数据 (JSON 格式)
    data = {
        "temperature": temp,
        "humidity": humi
    }

    try:
        # 3. 发送给服务器 (这行代码等同于 ESP32 发请求)
        response = requests.post(url, json=data)

        # 4. 打印结果
        if response.status_code == 200:
            print(f"✅ 发送成功: 温度={temp}°C, 湿度={humi}% | 服务器回复: {response.json()['message']}")
        else:
            print(f"❌ 发送失败: {response.status_code}")

    except Exception as e:
        print(f"❌ 连接错误: {e}")
        print("请检查: 1. Django 服务器开启了吗? 2. 地址对不对? ")
        break

    # 5. 休息 2 秒再发下一次
    time.sleep(2)
```

第二步：运行模拟器

这里有个**关键点**：你需要**同时运行两个**黑色窗口。

1. **窗口 A (服务器)**：
   - 这就是你刚才开着的那个 `python manage.py runserver` 的窗口。
   - **千万不要关掉它**，也不要动它。它是接收方，必须一直在线。
2. **窗口 B (模拟器)**：
   - 请**新开一个**命令行窗口 (CMD 或 PowerShell)。
   - 确保你也进入了 `IoT_System` 文件夹 (`cd ...`).
   - 运行模拟脚本：

     ```
     python simulate_device.py
     ```

第三步：见证奇迹

1. **观察窗口 B**：你应该会看到一行行绿色的字：`✅ 发送成功: 温度=23.5°C...` `✅ 发送成功: 温度=25.1°C...` 这就说明数据已经成功进入了你的数据库！
2. **回到浏览器**：刷新刚才那个 `http://127.0.0.1:8000/api/history/` 页面。

   原本是 `[]` 的地方，现在应该变成了满满的数据！ 类似这样：

```
{"data": [
    {"temp": 25.1, "humi": 50.2, "time": "20:30:05"},
    {"temp": 23.5, "humi": 48.9, "time": "20:30:03"}
]}
```

🟢 第四阶段：搭建 React 前端网页

我们要用一个叫 **Vite** 的工具来创建 React 项目，因为它比传统的工具快很多。

第一步：创建前端项目框架

请打开一个新的命令行窗口 (CMD)，先进入你的项目文件夹：

```
cd /d C:\Users\Gao\Desktop\毕业设计论文和方案\IoT_System
```

然后，执行下面的命令（**注意：** 如果它问你 `Ok to proceed? (y)`，请输入 `y` 并回车）：

```
npm create vite@latest frontend -- --template react
```

- 这会在文件夹里创建一个新目录叫 `frontend`。

接下来，进入这个新文件夹并安装必要的零件：

```
cd
npm install
```

- `npm install` 会根据清单下载 React 的核心代码，可能会花几十秒。

第二步：安装图表和网络工具

我们需要两个额外的工具：

1. **Axios**: 用来像 Python 的 `requests` 一样发送网络请求，去后端拿数据。
2. **Recharts**: 一个非常流行的 React 图表库，用来画温湿度曲线。

在 `frontend` 文件夹下，执行：

```
npm install axios recharts
```

第三步：编写网页代码 (App.jsx)

现在我们来写代码。请在编辑器中打开 `IoT_System/frontend/src/App.jsx` 文件。 **清空**里面的所有内容，复制粘贴下面的代码：

```
import { useState, useEffect } from 'react'
import axios from 'axios'
import { LineChart, Line, XAxis, YAxis, CartesianGrid, Tooltip, Legend,
ResponsiveContainer } from 'recharts'
import './App.css'

function App() {
  const [data, setData] = useState([])
```

```
  const [currentTemp, setCurrentTemp] = useState(0)
  const [currentHumi, setCurrentHumi] = useState(0)

  // 这个函数负责去后端拿数据
  const fetchData = () => {
    axios.get('http://127.0.0.1:8000/api/history/')
      .then(response => {
        const historyData = response.data.data
        // 如果拿到了数据
        if (historyData.length > 0) {
          // 1. 更新图表数据 (把原来的倒序 array 翻转一下，变成正序，方便画图从左到右)
          setData([...historyData].reverse())

          // 2. 更新当前显示的最新数值 (取第一个，因为后端发来的是最新的在前面)
          setCurrentTemp(historyData[0].temp)
          setCurrentHumi(historyData[0].humi)
        }
      })
      .catch(error => {
        console.error("哎呀，获取数据失败了:", error)
      })
  }

  // useEffect 相当于 "网页加载完毕后自动执行"
  useEffect(() => {
    // 1. 刚打开网页时，先拿一次数据
    fetchData()

    // 2. 之后每隔 2 秒钟自动去刷新一次数据 (轮询)
    const interval = setInterval(fetchData, 2000)

    // 清理工作 (当网页关闭时停止刷新)
    return () => clearInterval(interval)
  }, [])

  return (
    <div style={{ padding: '20px', fontFamily: 'Arial, sans-serif' }}>
      <h1>🌡️ 物联网实时监控系统</h1>

      {/* 实时数据显示卡片 */}
      <div style={{ display: 'flex', gap: '20px', marginBottom: '30px' }}>
        <div style={cardStyle}>
          <h3>当前温度</h3>
          <p style={{ fontSize: '40px', color: '#ff7300', margin: '10px 0' }}>
{currentTemp}°C</p>
        </div>
        <div style={cardStyle}>
          <h3>当前湿度</h3>
          <p style={{ fontSize: '40px', color: '#387908', margin: '10px 0' }}>
{currentHumi}%</p>
        </div>
      </div>

      {/* 历史趋势图 */}
      <div style={{ width: '100%', height: 400, background: '#f5f5f5',
borderRadius: '10px', padding: '20px' }}>
```

```
        <h3>📊 历史趋势图</h3>
        <ResponsiveContainer width="100%" height="100%">
          <LineChart data={data}>
            <CartesianGrid strokeDasharray="3 3" />
            <XAxis dataKey="time" />
            <YAxis />
            <Tooltip />
            <Legend />
            <Line type="monotone" dataKey="temp" stroke="#ff7300" name="温度 (°C)"
strokeWidth={3} />
            <Line type="monotone" dataKey="humi" stroke="#387908" name="湿度 (%)"
strokeWidth={3} />
          </LineChart>
        </ResponsiveContainer>
      </div>
    </div>
  )
}

// 一些简单的样式配置
const cardStyle = {
  border: '1px solid #ddd',
  borderRadius: '8px',
  padding: '20px',
  width: '200px',
  textAlign: 'center',
  boxShadow: '0 2px 5px rgba(0,0,0,0.1)'
}

export default App
```

第四步：启动前端！

在命令行窗口（frontend 文件夹下）输入：

```
npm run dev
```

你会看到类似这样的输出：`➜  Local:   http://localhost:5173/`

现在，按住 `Ctrl` 键，点击那个 `http://localhost:5173/` 链接（或者手动复制到浏览器打开）。

🎉 见证最终成果

打开网页后，你应该能看到：

1. **大大的数字**：显示着当前的温度和湿度。
2. **动态的图表**：每隔 2 秒钟，随着 Python 模拟脚本生成新数据，网页上的数字会跳动，图表的线条也会自动延伸！

如果看到图表在动，恭喜你！你的毕业设计原型机已经彻底跑通了！ 🚀

如果有任何报错，截图发给我。

git管理

arduino部分